EXAMEN DE SESSION NORMALE (SN) - SUPÉRIEUR

Session Normale (SN) - GI 101 : Algorithmique et Langage C (DUT Génie Informatique L1) - Année 2023-2024

Épreuve : Algorithmique et Programmation C | Niveau : DUT 1ère année | Durée : 3h00 | Barème : 20 pts

CONSIGNES GÉNÉRALES : La clarté, la rigueur de la démarche et la précision du vocabulaire académique sont impératives.

PARTIE 1 : ARCHITECTURE SYSTÈME, BINAIRE & OPÉRATEURS BITWISE (6 points)

Programmation bas niveau de microcontrôleurs industriels au laboratoire de génie informatique.

```
unsigned char reg = 0x3A; // 0011 1010 en binaire
reg |= (1 << 2);          // Mise a 1 du bit 2
reg &= ~(1 << 5);         // Mise a 0 du bit 5
reg ^= 0x01;              // Inversion du bit 0
```

1. Manipulation de registres binaires :

a) Donner la valeur binaire et hexadécimale de la variable 'reg' après l'exécution de chaque ligne de code ci-dessus.

b) Écrire une macro C nommée CHECK_BIT(reg, n) qui retourne 1 si le n-ième bit de reg est à 1 et 0 sinon.

c) Expliquer le principe de la représentation des entiers négatifs en complément à deux sur 8 bits pour la valeur -42.

PARTIE 2 : ALGORITHMES SUR TABLEAUX DYNAMIQUES & MATRICES CREUSES (7 points)

Représentation compressée CSR (Compressed Sparse Row) pour le calcul haute performance sur cartes embarquées.

```
typedef struct CSRMatrix {
    int num_rows;
    int num_cols;
    int nnz;        // Nombre d'elements non nuls
    double *values; // Tableau des valeurs
    int *col_idx;   // Indices des colonnes
    int *row_ptr;   // Pointeurs de debut de lignes
} CSRMatrix;
```

1. Fonctions de manipulation de matrice creuse :

a) Écrire la fonction CSRMatrix* init_csr(int rows, int cols, int nnz) allouant dynamiquement les 3 tableaux.

b) Écrire la fonction void mat_vec_mul(const CSRMatrix *A, const double *x, double *y) réalisant le produit y = A * x.

c) Calculer le gain d'espace mémoire en octets pour une matrice 1000x1000 ne contenant que 5000 éléments non nuls par rapport à un tableau 2D standard double M[1000][1000].

PARTIE 3 : MINI-PROJET - TRAITEMENT SÉCURISÉ DE TRAMES RÉSEAU (7 points)

Décodage de paquets de capteurs IoT industriels transitant par le port série sous Linux.

1. Spécification de la trame :

Format de la trame reçue : '$START,<id_capteur>,<temperature>,<pression>,<checksum>*END'.

Exemple : '$START,CAP_04,28.75,1013.2,8B*END'.

2. Écriture du parser en langage C :

a) Écrire la fonction int parse_frame(const char *raw, char *id, float *temp, float *press) extrayant les données avec vérification de format.

b) Écrire la fonction unsigned char compute_checksum(const char *data) calculant le XOR de tous les octets entre '$START,' et ',<checksum>'.

c) Comment se prémunir contre les failles de dépassement de tampon (buffer overflow) lors de la manipulation de chaînes avec strcpy vs strncpy ?